

Apostila SQLite para Gráficos em Python

sqlite3 · pandas · matplotlib

Exemplos completos, comentados linha a linha

16 capítulos · gráficos reais gerados a partir de um banco SQLite de exemplo

Sumário

Capítulo 1 — Por que SQLite + Python para gráficos?

Capítulo 2 — Gráfico de barras (o mais usado)

Capítulo 3 — Linhas e séries temporais

Capítulo 4 — Pizza e rosca (participação do todo)

Capítulo 5 — Barras empilhadas e 100% empilhadas

Capítulo 6 — Médias móveis com window functions

Capítulo 7 — Dispersão e gráfico de bolhas

Capítulo 8 — Histograma e boxplot (distribuição)

Capítulo 9 — Mapa de calor (heatmap)

Capítulo 10 — Pareto (barras + linha acumulada)

Capítulo 11 — Pannel com vários gráficos (subplots)

Capítulo 12 — Área e faixas de variação

Capítulo 13 — Formatação profissional em português

Capítulo 14 — Tabelas grandes: ler em lotes e continuar de onde parou

Capítulo 15 — Desempenho, segurança e exportação

Capítulo 16 — Referência rápida

Capítulo 1 — Por que SQLite + Python para gráficos?

O SQLite é um banco de dados que vive dentro de um único arquivo (loja.db), não exige servidor, instalação nem senha, e já vem embutido no Python pelo módulo `sqlite3`. Isso o torna a escolha ideal para análises locais e geração de gráficos: você guarda os dados brutos no banco, deixa o SQL fazer o trabalho pesado de filtrar, agrupar e somar, e entrega ao Python apenas o resultado já agregado — pequeno, limpo e pronto para virar gráfico. A regra de ouro desta apostila: o SQL agrega, o pandas organiza, o matplotlib desenha. Quanto mais cálculo você empurra para o SQL, menos memória o Python consome e mais rápido o gráfico aparece.

A ponte entre banco e gráfico

Todo exemplo desta apostila segue exatamente estas quatro linhas. Guarde esse esqueleto: conectar, consultar em DataFrame, plotar e salvar.

```
import sqlite3                # módulo do SQLite que já vem no Python
import pandas as pd           # organiza o resultado em tabela (DataFrame)
import matplotlib.pyplot as plt # biblioteca que desenha os gráficos

con = sqlite3.connect("loja.db") # abre (ou cria) o arquivo do banco
sql = "SELECT canal, COUNT(*) AS n FROM venda GROUP BY canal" # SQL já agregado
df = pd.read_sql_query(sql, con) # executa o SQL e devolve um DataFrame
con.close()                    # fecha a conexão: libera o arquivo

fig, ax = plt.subplots(figsize=(6,3)) # cria figura e eixo (a "folha" e o "desenho")
ax.bar(df["canal"], df["n"])          # barras: eixo X = canal, altura = contagem
ax.set_title("Vendas por canal")      # título do gráfico
fig.tight_layout()                  # ajusta margens para nada ficar cortado
fig.savefig("grafico.png", dpi=150)  # grava a imagem em disco (150 pontos/polegada)
```

O banco de exemplo usado no livro

Todos os gráficos a seguir saem deste mesmo banco, criado uma única vez. Ele simula uma loja com 24 meses de vendas, além de uma tabela de sensor para os gráficos de série temporal.

```
CREATE TABLE categoria (id INTEGER PRIMARY KEY, nome TEXT);           -- 5 categorias
CREATE TABLE produto   (id INTEGER PRIMARY KEY, nome TEXT,
                        categoria_id INT, preco REAL);                 -- 30 produtos
CREATE TABLE cliente   (id INTEGER PRIMARY KEY, nome TEXT,
                        cidade TEXT, uf TEXT);                         -- 120 clientes
CREATE TABLE venda     (id INTEGER PRIMARY KEY, data TEXT,
                        cliente_id INT, canal TEXT);                   -- 1 linha por pedido
CREATE TABLE item      (id INTEGER PRIMARY KEY, venda_id INT,
                        produto_id INT, qtd INT, preco REAL);          -- itens do pedido
CREATE TABLE sensor    (id INTEGER PRIMARY KEY, data TEXT,
                        temperatura REAL, umidade REAL);              -- 365 leituras
-- Observação: no SQLite a data é guardada como TEXT no formato 'YYYY-MM-DD'.
-- Esse formato ordena corretamente e funciona com as funções date()/strftime().
```

Capítulo 2 — Gráfico de barras (o mais usado)

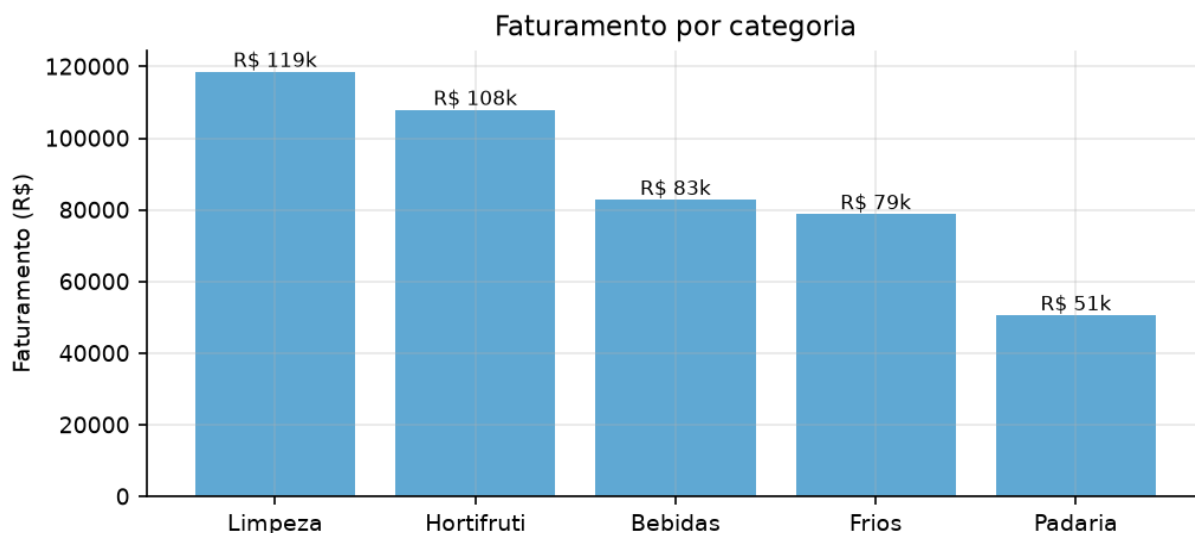
Barras servem para comparar categorias. No SQL isso é sempre a mesma receita: GROUP BY na coluna que vira o eixo X e uma função de agregação (SUM, COUNT, AVG) que vira a altura da barra. O ORDER BY no SQL já define a ordem das barras — não precisa reordenar no Python.

Barras verticais com rótulo de valor

Note que o SQL faz o JOIN de três tabelas para chegar na categoria, mas devolve apenas 5 linhas. É isso que queremos: o Python recebe pouca coisa.

```
SELECT c.nome AS categoria,          -- rótulo que vai no eixo X
       ROUND(SUM(i.qtd * i.preco), 2) AS faturamento -- altura da barra (qtd x preço)
FROM item i                          -- tabela dos itens vendidos
JOIN produto p ON p.id = i.produto_id -- traz o produto de cada item
JOIN categoria c ON c.id = p.categoria_id -- traz a categoria do produto
GROUP BY c.nome                     -- uma linha (barra) por categoria
ORDER BY faturamento DESC;         -- maior primeiro: barra ordenada

# ----- Python -----
df = pd.read_sql_query(sql, con)      # DataFrame com 5 linhas
fig, ax = plt.subplots(figsize=(7, 3.2)) # figura de 7x3,2 polegadas
ax.bar(df.categoria, df.faturamento, color="#5fa8d3") # desenha as barras
for x, v in zip(df.categoria, df.faturamento):        # percorre cada barra
    ax.text(x, v, f"R$ {v/1000:.0f}k",                # escreve o valor em milhares
            ha="center", va="bottom", fontsize=8)      # centralizado logo acima da barra
ax.set_ylabel("Faturamento (R$)")                    # nome do eixo Y
ax.set_title("Faturamento por categoria")              # título
```



Resultado gerado pelo código acima

Barras horizontais (barh) para nomes longos

Quando os rótulos são longos (nomes de produto, cidades), use barh: o texto fica legível sem girar. Aqui usamos ORDER BY total crescente porque o barh desenha de baixo para cima.

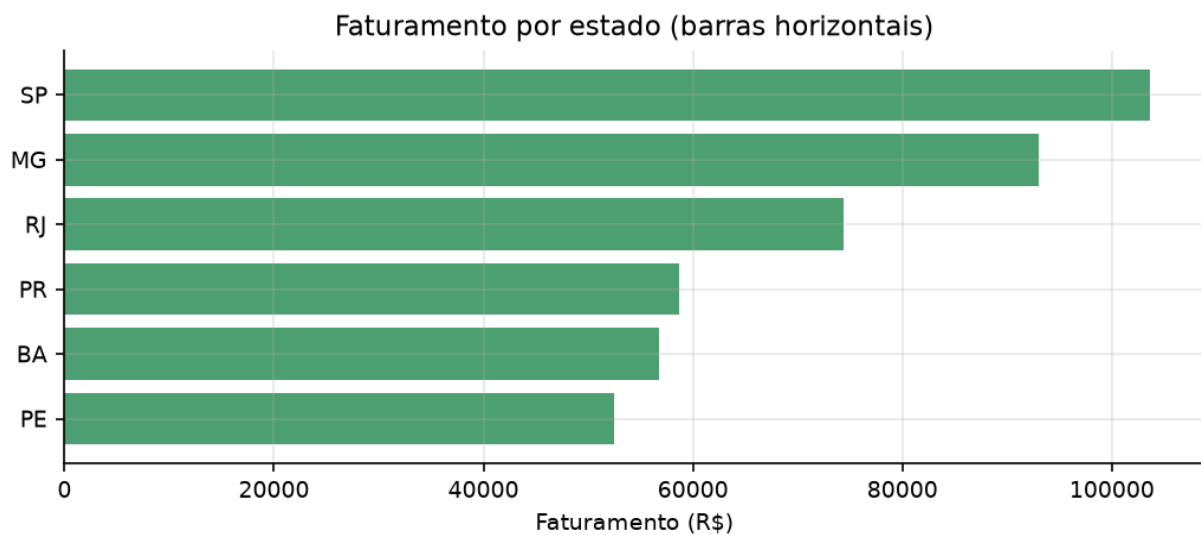
```

SELECT cl.uf,
       ROUND(SUM(i.qtd * i.preco), 2) AS total
FROM venda v
JOIN cliente cl ON cl.id = v.cliente_id
JOIN item i ON i.venda_id = v.id
GROUP BY cl.uf
ORDER BY total;

# ----- Python -----
ax.barh(df.uf, df.total, color="#4c9f70")
ax.set_xlabel("Faturamento (R$)")

```

-- estado do cliente = rótulo
-- comprimento da barra
-- liga a venda ao cliente
-- liga a venda aos seus itens
-- agrupa por estado
-- crescente: maior fica no topo do barh
barras deitadas (X = valor)
agora o valor está no eixo X



Resultado gerado pelo código acima

Capítulo 3 — Linhas e séries temporais

Linha é o gráfico da evolução no tempo. O segredo está na função `strftime('%Y-%m', data)` do SQLite, que recorta a data em mês, semana ou ano. Como a data é texto no padrão YYYY-MM-DD, o resultado de `strftime` ordena alfabeticamente na ordem cronológica correta — por isso um simples `ORDER BY mes` basta.

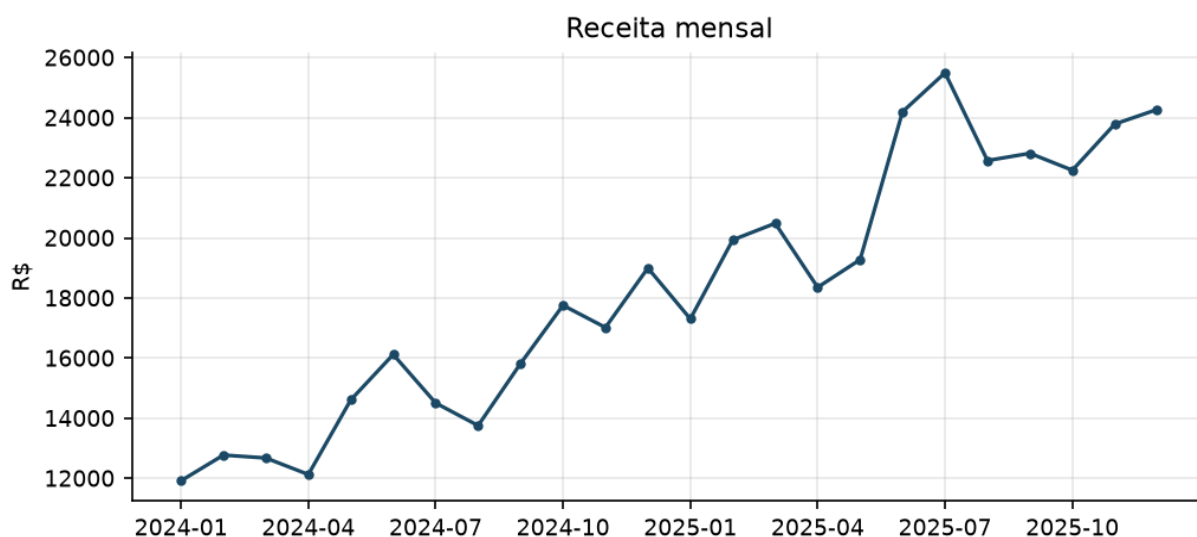
Uma linha: receita mês a mês

Formatos úteis do `strftime`: %Y ano, %m mês, %d dia, %W semana do ano, %w dia da semana (0=domingo), %H hora.

```
SELECT strftime('%Y-%m', v.data) AS mes,      -- recorta '2025-03-14' -> '2025-03'
       ROUND(SUM(i.qtd * i.preco), 2) AS receita -- soma o valor de todos os itens do mês
FROM venda v
JOIN item i ON i.venda_id = v.id             -- cada venda tem vários itens
GROUP BY mes                                 -- um ponto da linha por mês
ORDER BY mes;                                -- ordem cronológica (texto YYYY-MM ordena certo)
```



```
# ----- Python -----
ax.plot(df.mes, df.receita,                  # X = meses, Y = receita
        marker="o", ms=3,                   # marcador redondo pequeno em cada ponto
        color="#1b4965")                   # cor da linha
ax.set_xticks(df.mes[::3])                  # mostra 1 rótulo a cada 3 meses (evita amontoar)
ax.set_ylabel("R$")                         # unidade no eixo Y
```



Resultado gerado pelo código acima

Várias linhas: uma série por canal

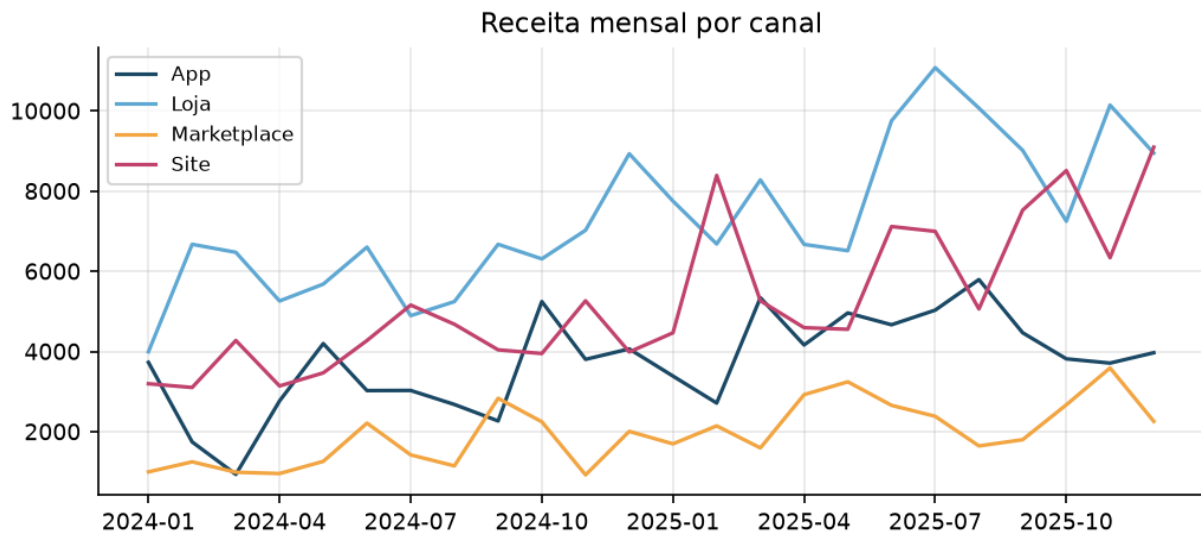
Para comparar séries, agrupe por duas colunas no SQL e use `pivot` no pandas: ele transforma a coluna de categoria em várias colunas, uma por linha do gráfico.

```

SELECT strftime('%Y-%m', v.data) AS mes, -- eixo X
       v.canal, -- vira UMA linha do gráfico para cada valor
       SUM(i.qtd * i.preco) AS receita -- eixo Y
FROM venda v JOIN item i ON i.venda_id = v.id
GROUP BY mes, v.canal -- agrupa pelas duas dimensões
ORDER BY mes;

# ----- Python -----
pv = df.pivot(index="mes",
              columns="canal",
              values="receita")
# linhas do quadro = meses
# colunas = canais (uma série cada)
# valores das células
for k, c in enumerate(pv.columns):
    # percorre cada canal
    ax.plot(pv.index, pv[c], label=c,
           color=PAL[k])
    # desenha a série com o nome na legenda
    # cor diferente por série
ax.legend(fontsize=8)
# exibe a legenda

```



Resultado gerado pelo código acima

Capítulo 4 — Pizza e rosca (participação do todo)

Use pizza apenas quando as fatias somam 100% de alguma coisa e existem poucas categorias (até 5 ou 6). Acima disso, prefira barras: o olho humano compara comprimentos muito melhor do que ângulos. Uma boa prática no SQL é agrupar a cauda longa em "Outros".

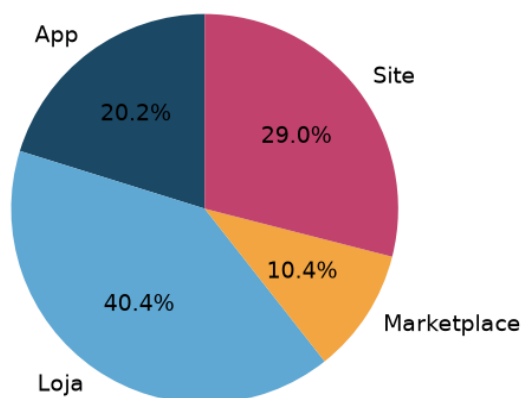
Pizza e rosca lado a lado

A rosca (donut) é a mesma pizza com `wedgeprops=dict(width=...)`: o buraco no meio deixa espaço para escrever o total.

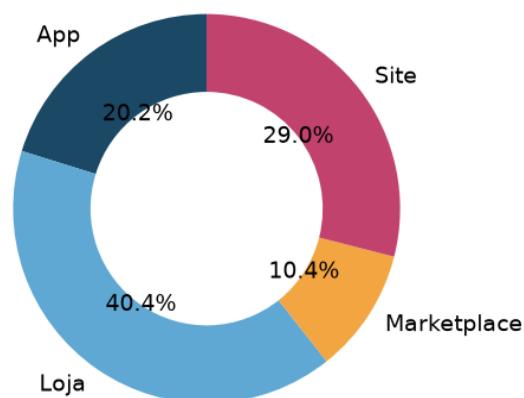
```
SELECT v.canal,          -- rótulo de cada fatia
       COUNT(*) AS pedidos -- tamanho da fatia = número de pedidos
FROM venda v
GROUP BY v.canal;        -- uma fatia por canal

# ----- Python -----
fig, axs = plt.subplots(1, 2, figsize=(7, 3.2)) # 1 linha, 2 colunas de gráficos
axs[0].pie(df.pedidos,                          # valores (o matplotlib calcula os % sozinho)
           labels=df.canal,                     # texto ao lado de cada fatia
           autopct="%1.1f%%",                  # escreve o percentual com 1 casa decimal
           colors=PAL,                         # lista de cores
           startangle=90)                      # começa no topo, sentido anti-horário
axs[1].pie(df.pedidos, labels=df.canal, autopct="%1.1f%%",
           colors=PAL, startangle=90,
           wedgeprops=dict(width=0.4)) # espessura do anel -> vira rosca
```

Pizza — participação por canal



Rosca (donut)



Resultado gerado pelo código acima

Agrupando a cauda longa em 'Outros'

Truque de SQL muito útil: um CASE junto de uma subconsulta com ranking transforma as categorias pequenas em uma única fatia, mantendo a pizza legível.


```

WITH tot AS (                                     -- CTE: total por produto
    SELECT p.nome, SUM(i.qtd * i.preco) AS v
    FROM item i JOIN produto p ON p.id = i.produto_id
    GROUP BY p.nome
), rk AS (                                       -- CTE: posiciona cada produto
    SELECT nome, v,
           ROW_NUMBER() OVER (ORDER BY v DESC) AS pos -- 1 = maior faturamento
    FROM tot
)
SELECT CASE WHEN pos <= 5 THEN nome              -- os 5 maiores mantêm o nome
           ELSE 'Outros' END AS fatia,          -- o resto vira 'Outros'
       SUM(v) AS valor                          -- soma dentro de cada fatia
FROM rk
GROUP BY fatia
ORDER BY valor DESC;

```

Capítulo 5 — Barras empilhadas e 100% empilhadas

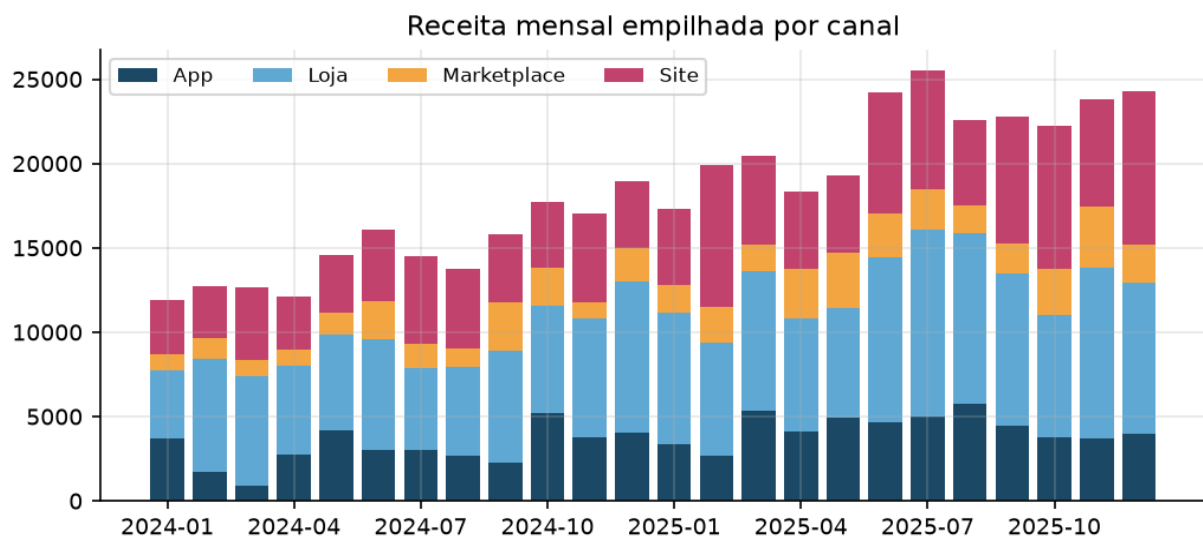
A barra empilhada mostra total e composição ao mesmo tempo. A versão 100% empilhada descarta o total e foca só na participação relativa — ideal para responder "o app está ganhando espaço frente à loja?".

Empilhamento por soma acumulada

No matplotlib não existe "empilhar automático": você mesmo passa o parâmetro `bottom`, que é a soma de tudo que já foi desenhado abaixo.

```
SELECT strftime('%Y-%m', v.data) AS mes, -- eixo X
       v.canal, -- cada canal é uma camada da pilha
       SUM(i.qtd * i.preco) AS r -- altura daquela camada
FROM venda v JOIN item i ON i.venda_id = v.id
GROUP BY mes, v.canal;

# ----- Python -----
pv = df.pivot(index="mes", columns="canal", values="r").fillna(0) # 0 onde faltou dado
base = 0 # começa a pilha no chão
for k, c in enumerate(pv.columns): # uma camada por canal
    ax.bar(pv.index, pv[c], # altura só desta camada
           bottom=base, # onde a camada começa (topo da anterior)
           label=c, color=PAL[k])
    base = base + pv[c] # novo topo acumulado
ax.legend(fontsize=8, ncol=4) # legenda em 4 colunas, ocupa menos espaço
```

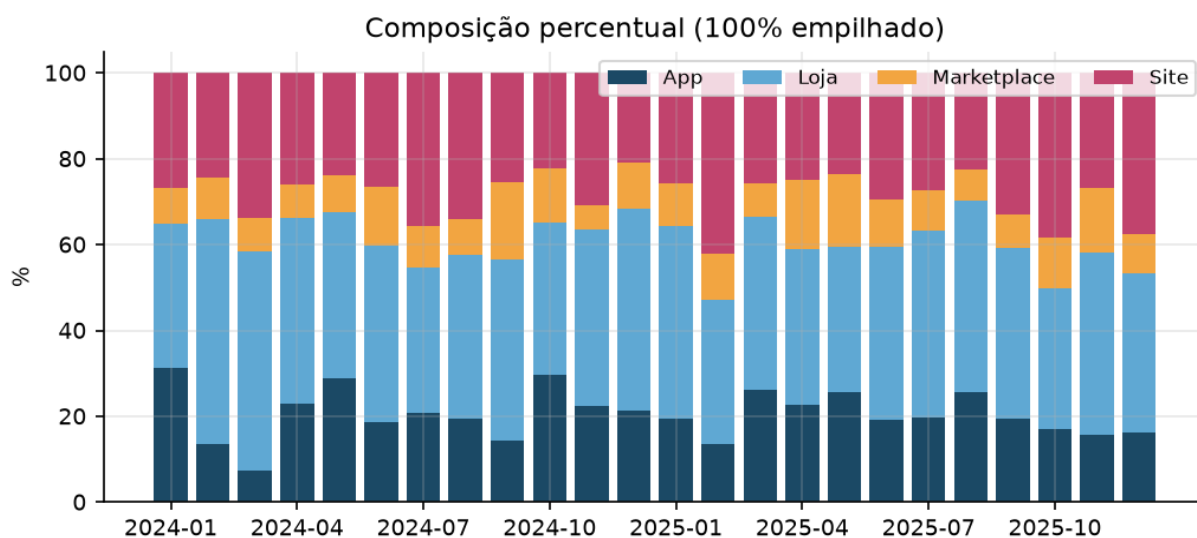


Resultado gerado pelo código acima

Normalizando para 100%

Basta dividir cada linha pelo total daquela linha. Tudo continua igual, só muda a escala.

```
pct = pv.div(pv.sum(axis=1), axis=0) * 100 # divide cada célula pelo total da linha
# pv.sum(axis=1) -> soma de cada mês; axis=0 alinha a divisão por linha
ax.set_ylabel("%") # agora o eixo Y vai de 0 a 100
```



Resultado gerado pelo código acima

Capítulo 6 — Médias móveis com window functions

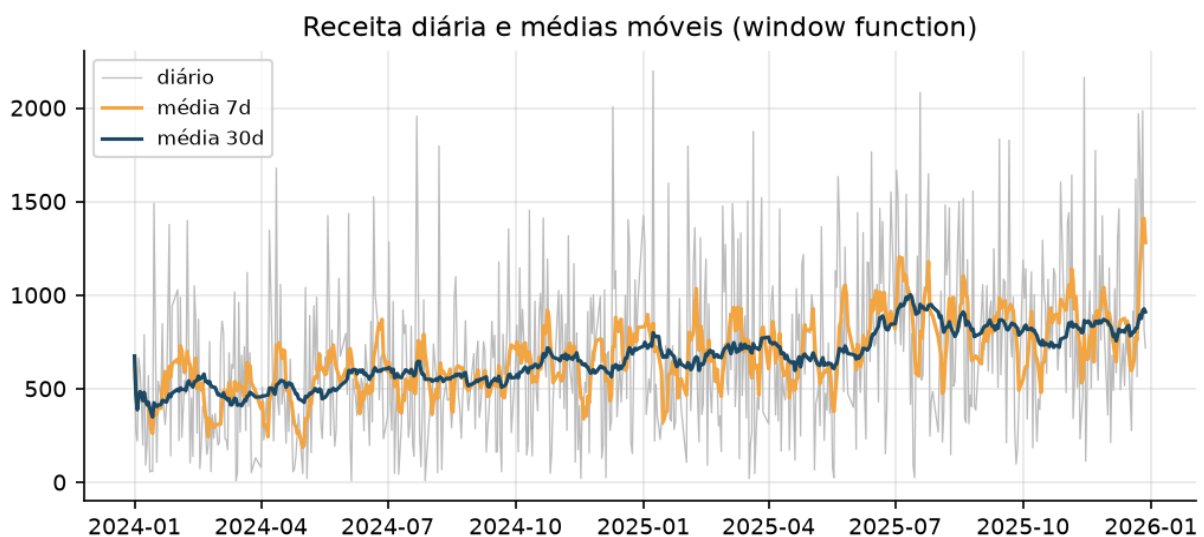
O SQLite suporta funções de janela desde a versão 3.25. Elas calculam algo "olhando para as linhas vizinhas" sem agrupar o resultado — perfeito para suavizar séries ruidosas antes de plotar. A cláusula `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` define a janela de 7 dias.

Média móvel de 7 e 30 dias

Repare que o ruído diário fica em cinza fino ao fundo e as médias mostram a tendência real. Sempre plote o dado bruto junto: ele dá contexto à linha suavizada.

```
WITH d AS (                                     -- 1º passo: total por dia
  SELECT v.data AS dia, SUM(i.qtd * i.preco) AS r
  FROM venda v JOIN item i ON i.venda_id = v.id
  GROUP BY v.data
)
SELECT dia, r,                                  -- valor bruto do dia
  AVG(r) OVER (ORDER BY dia                     -- janela percorre os dias em ordem
    ROWS BETWEEN 6 PRECEDING                  -- pega as 6 linhas anteriores...
    AND CURRENT ROW) AS mm7,                 -- ...mais a atual = média de 7 dias
  AVG(r) OVER (ORDER BY dia
    ROWS BETWEEN 29 PRECEDING
    AND CURRENT ROW) AS mm30                -- média de 30 dias
FROM d
ORDER BY dia;

# ----- Python -----
x = pd.to_datetime(df.dia)                    # converte texto em data de verdade
ax.plot(x, df.r, lw=.6, color="#bbbbbb", label="diário") # linha fina = ruído
ax.plot(x, df.mm7, color="#f2a541", label="média 7d")   # tendência curta
ax.plot(x, df.mm30, color="#1b4965", label="média 30d") # tendência longa
```



Resultado gerado pelo código acima

Capítulo 7 — Dispersão e gráfico de bolhas

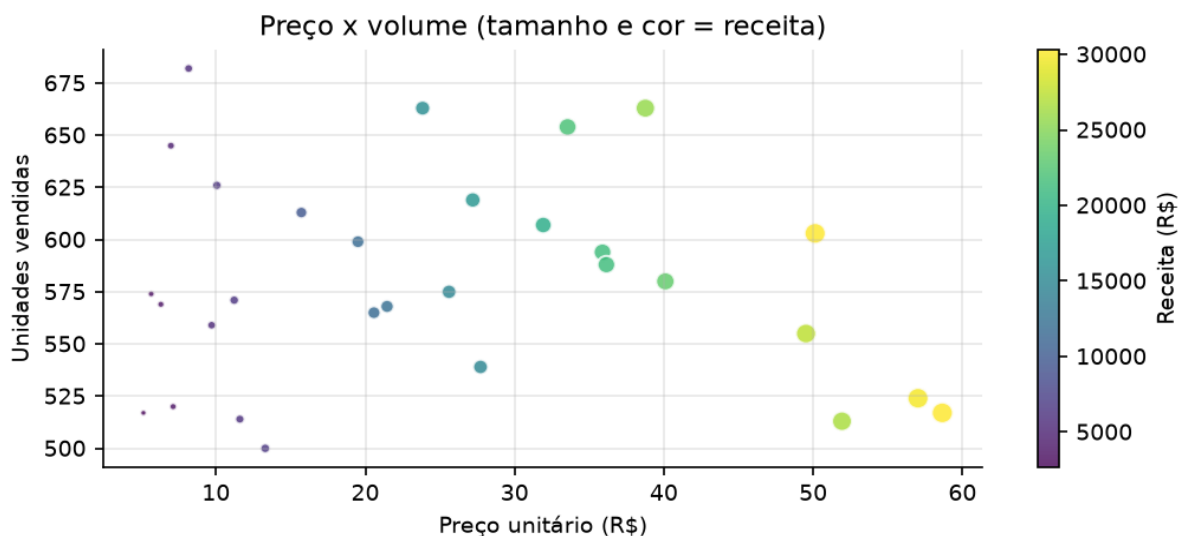
A dispersão (scatter) revela a relação entre duas medidas numéricas. Adicionando os parâmetros *s* (tamanho) e *c* (cor) você mostra quatro variáveis em um único desenho — é o chamado gráfico de bolhas.

Preço x unidades x receita

O SQL devolve uma linha por produto com três medidas; cada linha vira um ponto. Divida o valor usado em *s* por uma constante até as bolhas ficarem proporcionais.

```
SELECT p.nome,                -- identificação do ponto
       p.preco,                -- eixo X
       SUM(i.qtd)              AS unidades, -- eixo Y
       SUM(i.qtd * i.preco)    AS receita   -- tamanho e cor da bolha
FROM item i JOIN produto p ON p.id = i.produto_id
GROUP BY p.id;                -- um ponto por produto

# ----- Python -----
sc = ax.scatter(df.preco, df.unidades,      # posição do ponto
               s=df.receita / 400,         # área da bolha (dividida para caber)
               c=df.receita,               # cor mapeada pelo valor
               cmap="viridis",             # paleta contínua
               alpha=.8,                   # leve transparência: mostra sobreposição
               edgecolor="white")           # contorno branco separa bolhas coladas
fig.colorbar(sc, ax=ax, label="Receita (R$)") # barra de cor explicando a escala
```



Resultado gerado pelo código acima

Capítulo 8 — Histograma e boxplot (distribuição)

Média engana. Duas lojas com o mesmo ticket médio podem ter clientes completamente diferentes. O histograma mostra o formato da distribuição e o boxplot resume mediana, quartis e outliers — os dois respondem "como os valores se espalham?".

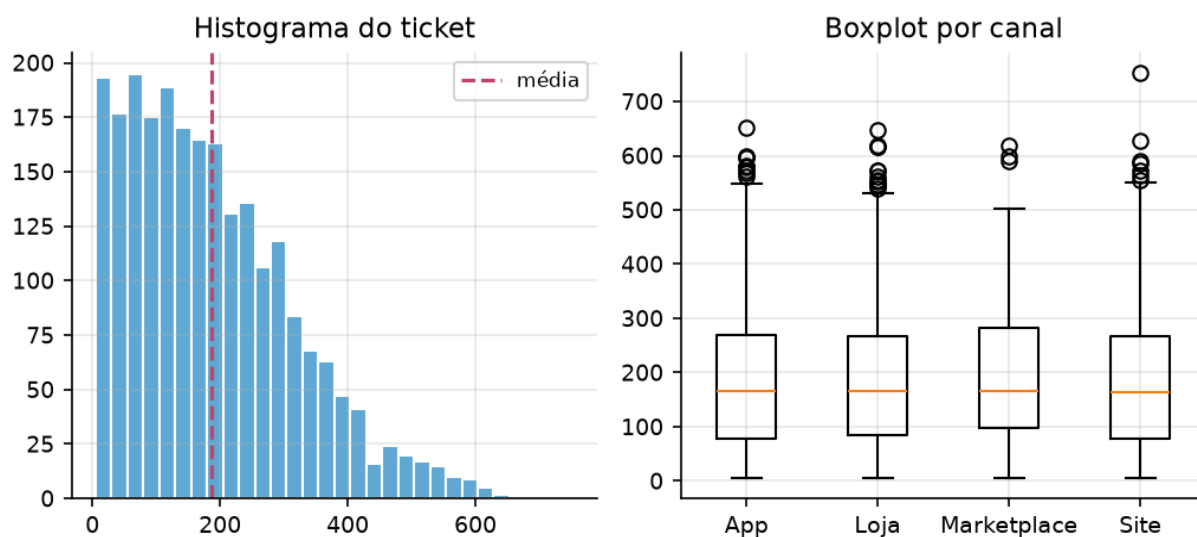
Ticket médio: distribuição e comparação

Aqui o SQL não agrega tudo: ele devolve uma linha por venda, porque o histograma precisa dos valores individuais. É a única situação em que trazemos muitas linhas.

```
SELECT v.id, -- uma linha por pedido
       SUM(i.qtd * i.preco) AS ticket -- valor total daquele pedido
FROM venda v JOIN item i ON i.venda_id = v.id
GROUP BY v.id; -- agrupa pelos itens de cada pedido

# ----- Python -----
# série de valores individuais
# 30 faixas (barras) no histograma
# linha vertical na média
# e os pontos soltos além dos "bigodes" são outliers.

# lista de arrays, um por canal
# desenha uma caixa por canal
# nomes no eixo X
# A caixa vai do 1º ao 3º quartil, a linha do meio é a MEDIANA
```



Resultado gerado pelo código acima

Capítulo 9 — Mapa de calor (heatmap)

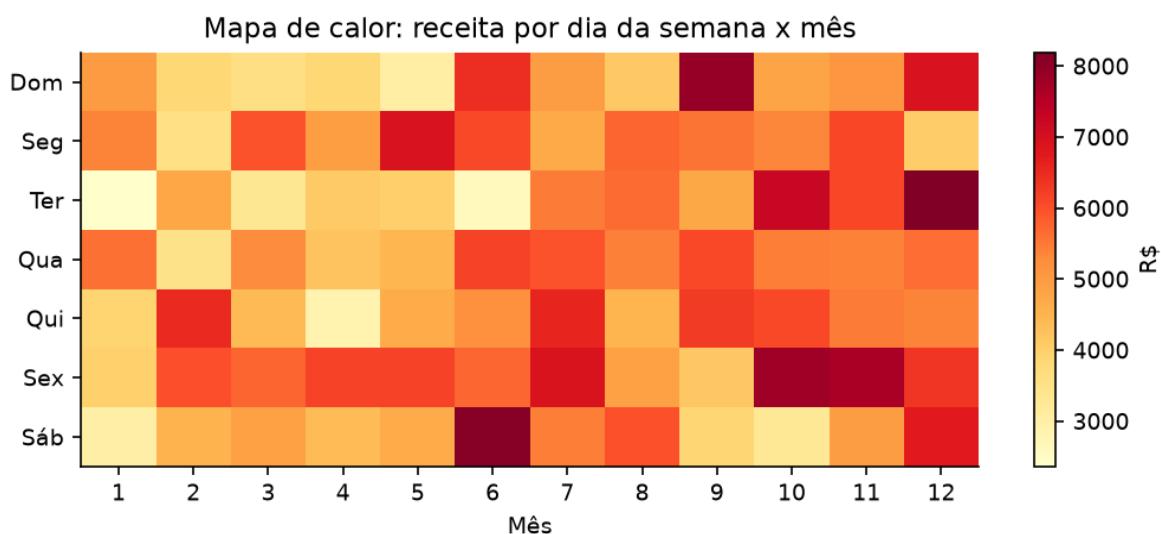
O heatmap cruza duas dimensões e pinta a intensidade de uma medida. É a forma mais rápida de achar padrões sazonais: "sexta-feira em dezembro vende muito mais". Sem bibliotecas extras: imshow do matplotlib já resolve.

Receita por dia da semana e mês

strftime('%w') devolve o dia da semana como texto ('0' a '6'); o CAST(... AS INT) converte para número para que o pivot fique ordenado corretamente.

```
SELECT CAST(strftime('%w', v.data) AS INT) AS dow, -- 0=domingo ... 6=sábado
       CAST(strftime('%m', v.data) AS INT) AS mes, -- 1..12
       SUM(i.qtd * i.preco) AS r -- intensidade da célula
FROM venda v JOIN item i ON i.venda_id = v.id
GROUP BY dow, mes; -- uma célula por cruzamento

# ----- Python -----
pv = df.pivot(index="dow", columns="mes", values="r").fillna(0) # matriz 7x12
im = ax.imshow(pv.values, # matriz numérica -> imagem
               cmap="YlOrRd", # amarelo (baixo) -> vermelho (alto)
               aspect="auto") # estica para preencher o eixo
ax.set_yticks(range(len(pv.index))) # posições dos rótulos
ax.set_yticklabels(["Dom", "Seg", "Ter", "Qua", "Qui", "Sex", "Sáb"])
ax.grid(False) # grade atrapalha em heatmap
fig.colorbar(im, ax=ax, label="R$") # legenda de cores
```



Resultado gerado pelo código acima

Capítulo 10 — Pareto (barras + linha acumulada)

O gráfico de Pareto combina barras ordenadas com uma linha do percentual acumulado em um segundo eixo Y. Serve para a regra 80/20: quantos produtos respondem por 80% da receita? O recurso-chave do matplotlib é o `twinx()`, que cria um eixo Y à direita.

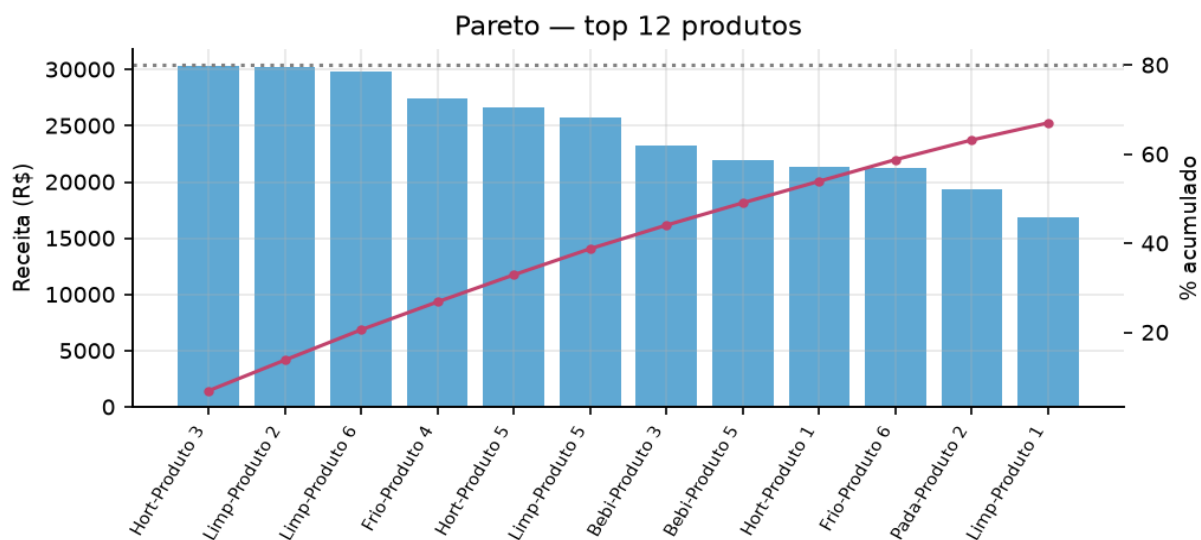
Top 12 produtos com curva acumulada

O acumulado pode ser feito no SQL com `SUM()` OVER (ORDER BY ...) ou no pandas com `cumsum()`. Abaixo mostro as duas formas.

```
-- Opção 1: tudo no SQL, com window function
SELECT nome, r,
       ROUND(100.0 * SUM(r) OVER (ORDER BY r DESC)
           / SUM(r) OVER (), 2) AS pct_acum
FROM (SELECT p.nome, SUM(i.qtd * i.preco) AS r
      FROM item i JOIN produto p ON p.id = i.produto_id
      GROUP BY p.id)
ORDER BY r DESC LIMIT 12;

# Opção 2: acumulado no pandas
df["acum"] = df.r.cumsum() / total * 100      # cumsum = soma acumulada

# ----- Python -----
ax.bar(range(len(df)), df.r, color="#5fa8d3") # barras na ordem decrescente
ax.set_xticklabels(df.nome, rotation=60,      # gira o texto 60 graus
                  ha="right", fontsize=7)    # alinhado à direita para não sobrepor
ax2 = ax.twinx()                             # SEGUNDO eixo Y, à direita
ax2.plot(range(len(df)), df.acum,           # linha do acumulado
         color="#c1436d", marker="o", ms=3)
ax2.axhline(80, color="gray", ls=":")      # referência dos 80%
ax2.grid(False)                             # evita grade dupla
```



Resultado gerado pelo código acima

Capítulo 11 — Painel com vários gráficos (subplots)

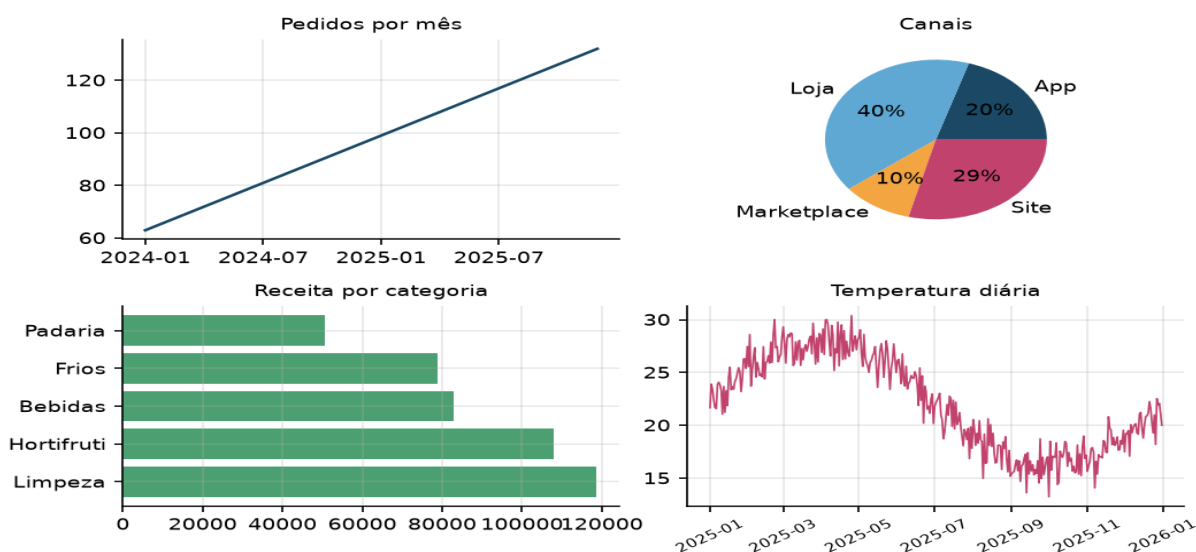
Um painel é apenas uma figura com uma grade de eixos. Você cria a grade com `plt.subplots(linhas, colunas)` e acessa cada quadro por índice, como uma matriz: `axs[0,0]`, `axs[0,1]`, e assim por diante. Cada quadro recebe sua própria consulta SQL.

Painel 2x2 com quatro consultas

Dica de desempenho: abra a conexão uma vez, rode as quatro consultas e só depois feche. Abrir e fechar o arquivo a cada consulta é o erro mais comum de lentidão com SQLite.

```
con = sqlite3.connect("loja.db") # abre UMA vez
d1 = pd.read_sql_query(sql_pedidos_mes, con) # consulta 1
d2 = pd.read_sql_query(sql_canais, con) # consulta 2
d3 = pd.read_sql_query(sql_categorias, con) # consulta 3
d4 = pd.read_sql_query(sql_sensor, con) # consulta 4
con.close() # fecha no fim

fig, axs = plt.subplots(2, 2, figsize=(7.2, 4.6)) # grade 2x2 de eixos
axs[0,0].plot(d1.m, d1.n) # quadro superior esquerdo
axs[0,1].pie(d2.n, labels=d2.canal) # superior direito
axs[1,0].barh(d3.nome, d3.r) # inferior esquerdo
axs[1,1].plot(pd.to_datetime(d4.data), d4.temperatura) # inferior direito
axs[1,1].tick_params(axis="x", rotation=30, # gira as datas
                    labels=7)
fig.tight_layout() # impede que os títulos se sobreponham
```



Resultado gerado pelo código acima

Capítulo 12 — Área e faixas de variação

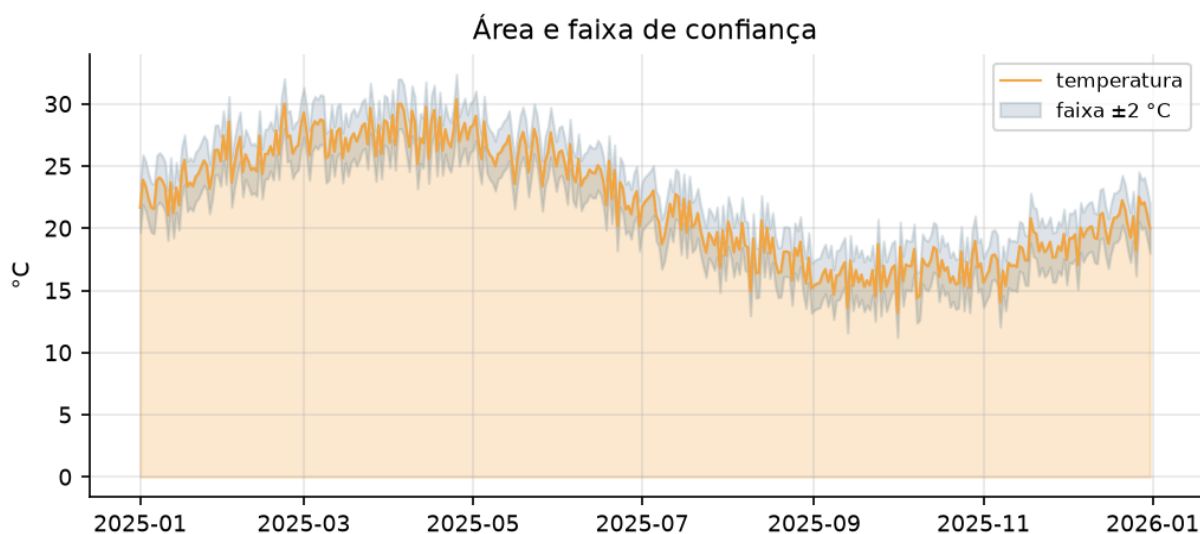
`fill_between` pinta a região entre duas curvas (ou entre a curva e o zero). Use para gráficos de área, faixas de erro, metas e intervalos mínimo/máximo — tudo que o SQL consegue devolver com `MIN()` e `MAX()` agregados.

Curva com área e faixa

A transparência (`alpha`) é essencial: sem ela a área tapa a linha.

```
SELECT data,          -- eixo X (texto YYYY-MM-DD)
       temperatura,   -- curva principal
       umidade        -- segunda medida disponível
FROM sensor
ORDER BY data;        -- sempre ordene séries temporais no SQL

# ----- Python -----
x = pd.to_datetime(df.data)          # texto -> data
ax.fill_between(x, df.temperatura,    # pinta do zero até a curva
               alpha=.25, color="#f2a541") # 25% de opacidade
ax.plot(x, df.temperatura, lw=1)      # desenha a linha por cima da área
ax.fill_between(x,
               df.temperatura - 2,    # limite inferior da faixa
               df.temperatura + 2,    # limite superior
               alpha=.15, label="faixa ±2 °C")
```



Resultado gerado pelo código acima

Faixa mínimo/máximo direto do SQL

Quando os limites vêm dos dados, deixe o SQL calcular por período.

```
SELECT strftime('%Y-%m', data) AS mes,    -- agrupa por mês
       MIN(temperatura) AS minimo,        -- limite inferior da faixa
       AVG(temperatura) AS media,         -- linha central
       MAX(temperatura) AS maximo         -- limite superior
FROM sensor
GROUP BY mes
ORDER BY mes;

# ax.fill_between(df.mes, df.minimo, df.maximo, alpha=.2) # faixa
# ax.plot(df.mes, df.media)                             # média
```

Capítulo 13 — Formatação profissional em português

Um gráfico bom é um gráfico legível: valores em reais, datas em português, título alinhado à esquerda e anotações apontando o que importa. Nada disso vem por padrão — mas são poucas linhas.

Reais no eixo, anotação e título alinhado

O `FuncFormatter` intercepta cada valor do eixo e devolve o texto que você quiser. O truque do `.replace(",", ".")` converte o separador de milhar americano no padrão brasileiro.

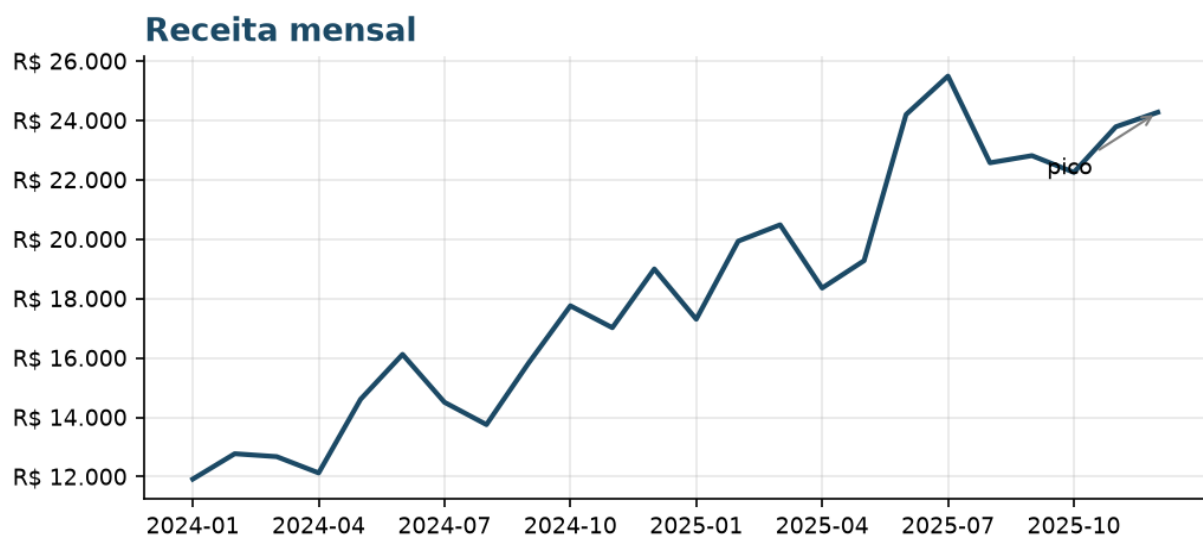
```
import matplotlib.ticker as mtick          # módulo dos formataadores de eixo

ax.yaxis.set_major_formatter(              # aplica no eixo Y
    mtick.FuncFormatter(                   # formatador com função própria
        lambda v, p: f"R$ {v:,.0f}"      # 1234567 -> 'R$ 1,234,567'
        .replace(",", ".")               # -> 'R$ 1.234.567' (padrão BR)
    ))

ax.set_title("Receita mensal",             # título maior e em negrito
    fontsize=13, fontweight="bold",        # alinhado à esquerda (padrão editorial)
    loc="left",
    color="#1b4965")

ax.annotate("pico",                       # texto da anotação
    xy=(x_do_ponto, y_do_ponto),          # ponto apontado
    xytext=(-45, -25),                   # deslocamento do texto em pontos
    textcoords="offset points",
    arrowprops=dict(arrowstyle="->",      # seta fina
        color="gray"))

# Outros ajustes úteis:
# ax.tick_params(axis="x", rotation=45)   # gira rótulos do eixo X
# ax.spines["top"].set_visible(False)     # remove a moldura de cima
# ax.grid(alpha=.3)                      # grade discreta
# plt.rcParams["font.size"] = 9          # tamanho padrão de todo texto
```



Resultado gerado pelo código acima

Capítulo 14 — Tabelas grandes: ler em lotes e continuar de onde parou

Quando a tabela não cabe na memória, não use `pd.read_sql_query` de uma vez. Leia em lotes (chunks), agregue o pedaço na hora e descarte-o. Duas técnicas: `chunksize` do pandas (simples) e `keyset pagination` com marcador salvo (retomável depois de uma queda).

Leitura em lotes com `chunksize`

O pandas devolve um iterador: cada volta do laço carrega apenas N linhas na memória.

```
acumulado = {} # guarda só o resultado agregado
sql = "SELECT strftime('%Y-%m', data) m, valor FROM movimento"
for pedaco in pd.read_sql_query(sql, con, # devolve um iterador, não um DataFrame
                               chunksize=50_000): # 50 mil linhas por vez
    parcial = pedaco.groupby("m")["valor"].sum() # agrega o lote
    for mes, v in parcial.items(): # soma no acumulador global
        acumulado[mes] = acumulado.get(mes, 0) + v
    # 'pedaco' é descartado aqui: a memória nunca cresce

serie = pd.Series(acumulado).sort_index() # vira a série final do gráfico
serie.plot(kind="bar") # e já pode ser plotada
```

Keyset pagination retomável (continua de onde parou)

Guarde o último id processado em uma tabela de controle. Se o script cair, ele recomeça exatamente do próximo registro — e nunca reprocessa nada. É muito mais rápido que `LIMIT/OFFSET`, que a cada página relê tudo o que já passou.

```
-- tabela de controle: guarda o marcador (bookmark)
CREATE TABLE IF NOT EXISTS progresso (
    tarefa TEXT PRIMARY KEY, -- nome do job, ex.: 'grafico_mensal'
    ultimo_id INTEGER NOT NULL -- último id já processado
);
CREATE INDEX IF NOT EXISTS ix_mov_id ON movimento(id); -- índice = paginação rápida

# ----- Python -----
TAREFA, LOTE = "grafico_mensal", 50_000
cur = con.cursor()
cur.execute("INSERT OR IGNORE INTO progresso VALUES (?, 0)", (TAREFA,)) # 1ª vez começa do 0
ultimo = cur.execute("SELECT ultimo_id FROM progresso WHERE tarefa=?",
                    (TAREFA,)).fetchone()[0] # retoma o marcador salvo

acumulado = {}
while True:
    df = pd.read_sql_query(
        "SELECT id, data, valor FROM movimento "
        "WHERE id > ? " # <- keyset: pega só o que vem DEPOIS do marcador
        "ORDER BY id LIMIT ?", # ordem pelo id garante que nada se repete
        con, params=(ultimo, LOTE))
    if df.empty: # acabou: não há mais linhas
        break
    parcial = df.assign(m=df.data.str[:7]).groupby("m").valor.sum() # agrega o lote
    for mes, v in parcial.items():
        acumulado[mes] = acumulado.get(mes, 0) + v

    ultimo = int(df.id.iloc[-1]) # novo marcador = último id lido
    cur.execute("UPDATE progresso SET ultimo_id=? WHERE tarefa=?", (ultimo, TAREFA))
    con.commit() # grava o marcador: se cair agora, retoma daqui

# Por que não LIMIT/OFFSET?
# OFFSET 1000000 obriga o banco a percorrer e descartar 1 milhão de linhas
# a cada página; com 'WHERE id > ?' o índice salta direto para a posição certa.
```

Cursor puro, sem pandas, para memória mínima

Quando nem o lote cabe na memória, itere linha a linha com o cursor do sqlite3.

```
cur = con.cursor()
cur.arraysize = 10_000 # nº de linhas que o driver busca por vez
cur.execute("SELECT data, valor FROM movimento ORDER BY id")
soma = {}
while True:
    linhas = cur.fetchmany(cur.arraysize) # busca o próximo bloco
    if not linhas: # lista vazia = fim
        break
    for data, valor in linhas: # processa e descarta
        m = data[:7] # 'YYYY-MM'
        soma[m] = soma.get(m, 0) + valor
```

Capítulo 15 — Desempenho, segurança e exportação

Três hábitos separam um script amador de um profissional: criar índices para as colunas usadas em filtros e agrupamentos, nunca concatenar valores dentro do SQL, e salvar a figura em formato adequado ao destino.

Índices: o que torna o gráfico instantâneo

Sem índice, cada consulta lê a tabela inteira. Com índice, o SQLite salta direto para as linhas relevantes. Crie um índice para cada coluna que aparece em WHERE, JOIN ou GROUP BY.

```
CREATE INDEX IF NOT EXISTS ix_venda_data ON venda(data);      -- filtros por período
CREATE INDEX IF NOT EXISTS ix_item_venda ON item(venda_id);   -- acelera o JOIN
CREATE INDEX IF NOT EXISTS ix_item_prod ON item(produto_id);  -- acelera o JOIN
CREATE INDEX IF NOT EXISTS ix_venda_canal ON venda(canal);    -- GROUP BY canal

ANALYZE;              -- coleta estatísticas: ajuda o otimizador a escolher o índice
PRAGMA journal_mode = WAL;  -- modo WAL: leitura e escrita simultâneas, mais rápido
PRAGMA synchronous = NORMAL; -- menos gravações no disco em cargas grandes

EXPLAIN QUERY PLAN      -- mostra COMO o SQLite vai executar a consulta
SELECT ... ;            -- se aparecer 'SCAN tabela', falta um índice
                        -- se aparecer 'SEARCH ... USING INDEX', está ótimo
```

Parâmetros: nunca use f-string no SQL

Concatenar texto abre a porta para injeção de SQL e quebra com aspas simples nos dados. Use sempre ? e passe os valores em params.

```
# ERRADO – vulnerável e frágil
sql = f"SELECT * FROM venda WHERE canal = '{canal}'"      # nunca faça isso

# CERTO – placeholder e parâmetros
df = pd.read_sql_query(
    "SELECT strftime('%Y-%m', data) m, COUNT(*) n "
    "FROM venda WHERE canal = ? AND data BETWEEN ? AND ? " # ? = espaço reservado
    "GROUP BY m ORDER BY m",
    con,
    params=(canal, inicio, fim)) # os valores viajam separados do texto do SQL

# Com o cursor puro é igual:
cur.execute("DELETE FROM venda WHERE id = ?", (venda_id,)) # tupla, mesmo com 1 item
```

Salvando a figura no formato certo

PNG para telas e relatórios, SVG/PDF para impressão (vetorial, não perde qualidade ao ampliar). E sempre feche a figura em laços, senão a memória cresce sem parar.

```

fig.tight_layout()                # ajusta margens antes de salvar
fig.savefig("grafico.png", dpi=150, # PNG: 150 dpi para tela, 300 para impressão
            bbox_inches="tight",    # corta o espaço branco em volta
            facecolor="white")      # fundo branco (evita PNG transparente)
fig.savefig("grafico.svg")         # vetorial: perfeito em qualquer tamanho
fig.savefig("grafico.pdf")         # vetorial, pronto para relatório
plt.close(fig)                    # libera a memória (obrigatório dentro de laços)

# Gerando um gráfico por categoria de uma vez:
for cat in df.categoria.unique(): # percorre as categorias
    sub = pd.read_sql_query(sql, con, params=(cat,)) # consulta filtrada
    fig, ax = plt.subplots()
    ax.plot(sub.mes, sub.receita)
    ax.set_title(f"Receita - {cat}")
    fig.savefig(f"saida/{cat}.png", dpi=150, bbox_inches="tight")
    plt.close(fig)                # fecha SEMPRE

```

Capítulo 16 — Referência rápida

Cole esta página na parede. À esquerda, o que você quer mostrar; à direita, a receita de SQL e a função do matplotlib.

Tabela de decisão

O que você quer mostrar	SQL	matplotlib
Comparar categorias	GROUP BY + SUM	ax.bar / ax.barh
Evolução no tempo	strftime('%Y-%m') + GROUP BY	ax.plot
Participação do todo	GROUP BY + COUNT	ax.pie
Total e composição	GROUP BY duas colunas + pivot	ax.bar(bottom=)
Tendência sem ruído	AVG() OVER (ROWS ...)	ax.plot
Relação entre medidas	GROUP BY com 2+ métricas	ax.scatter
Distribuição dos valores	SELECT sem agregar	ax.hist / ax.boxplot
Padrão em duas dimensões	GROUP BY dow, mes + pivot	ax.imshow
Regra 80/20	SUM() OVER (ORDER BY)	ax.bar + twinx
Faixa mín./máx.	MIN() e MAX()	ax.fill_between
Vários gráficos juntos	uma consulta por quadro	plt.subplots(2,2)

Funções de data do SQLite

Todas operam sobre texto no formato YYYY-MM-DD HH:MM:SS.

```
strftime('%Y', data) -- '2025'          ano
strftime('%Y-%m', data) -- '2025-03'    mês (o mais usado em gráficos)
strftime('%Y-%W', data) -- '2025-11'    semana do ano
strftime('%w', data) -- '0'..'6'        dia da semana (0 = domingo)
strftime('%H', data) -- '00'..'23'      hora
date(data, 'start of month') -- primeiro dia do mês
date(data, '-30 days') -- 30 dias atrás
date('now') -- hoje
julianday(fim) - julianday(ini) -- diferença em dias (número real)

-- Filtrar os últimos 12 meses:
WHERE data >= date('now', '-12 months')
```

Agregações e janelas mais usadas


```

COUNT(*)           -- quantidade de linhas
COUNT(DISTINCT cli) -- valores distintos (clientes únicos)
SUM(qtd * preco)     -- soma (altura da barra)
AVG(valor)           -- média
MIN(v) / MAX(v)      -- extremos (faixas em fill_between)
TOTAL(v)             -- como SUM, mas devolve 0.0 em vez de NULL quando vazio

-- Janelas (SQLite 3.25+):
SUM(v) OVER (ORDER BY data)           -- acumulado
AVG(v) OVER (ORDER BY data ROWS BETWEEN 6 PRECEDING
              AND CURRENT ROW)         -- média móvel 7
LAG(v) OVER (ORDER BY data)           -- valor anterior
100.0 * (v - LAG(v) OVER (ORDER BY data))
      / LAG(v) OVER (ORDER BY data)   -- variação %
RANK() OVER (PARTITION BY cat ORDER BY v DESC) -- ranking por grupo
ROW_NUMBER() OVER (ORDER BY v DESC)   -- posição 1,2,3...

```